ChatGPT 說：

SGGenAxis — 10 大節點 I/O 架構與 UI 整合規格

Design Specification · Architecture Inspector · Schema Validation · Dependency Graph

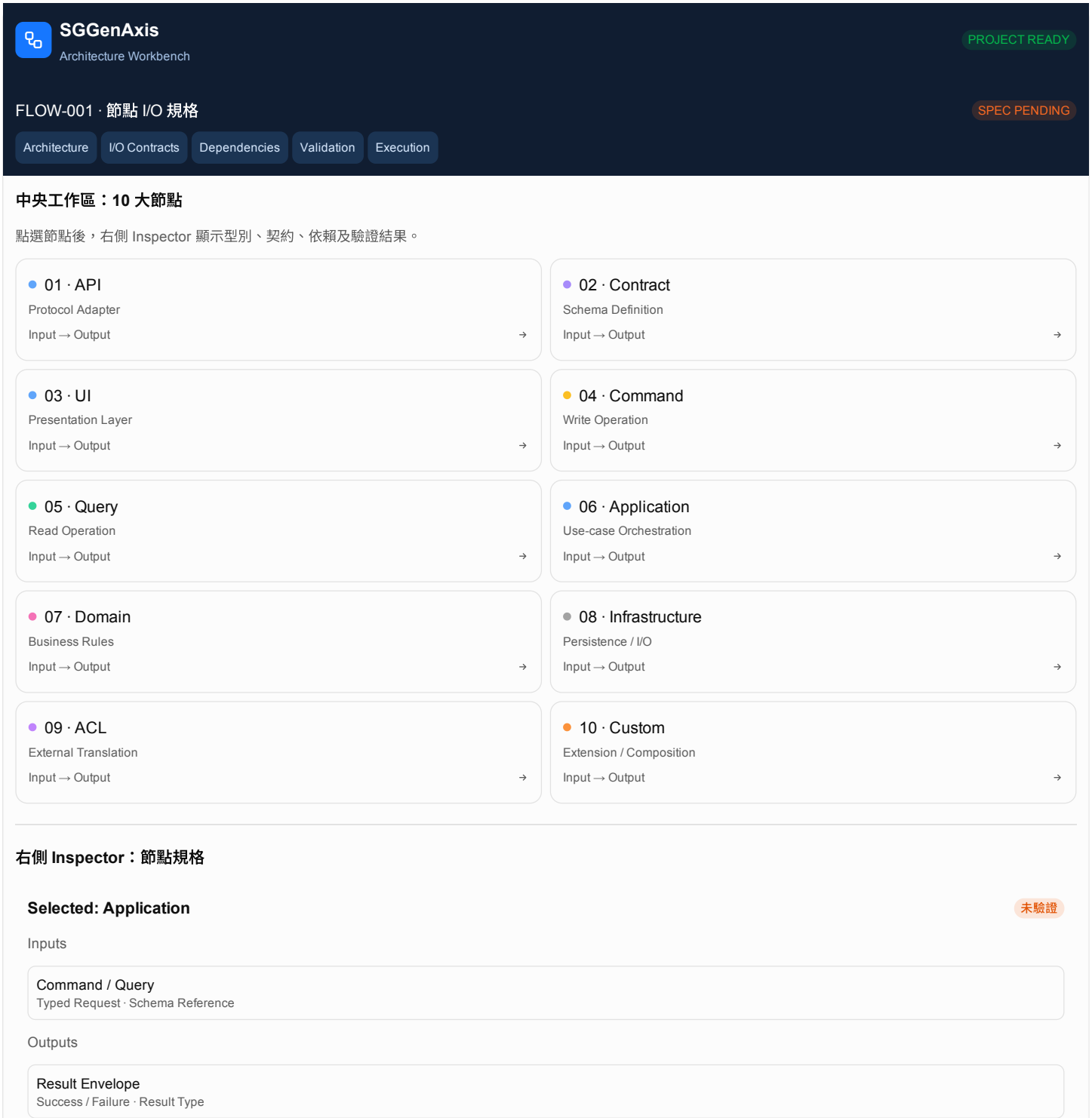
SGGenAxis 不應只是一個展示任務狀態的 Dashboard，而應進一步成為一套能夠**定義架構、檢查資料契約、驗證模組依賴，並將規格交給 Agent 執行**的工程管理平台。

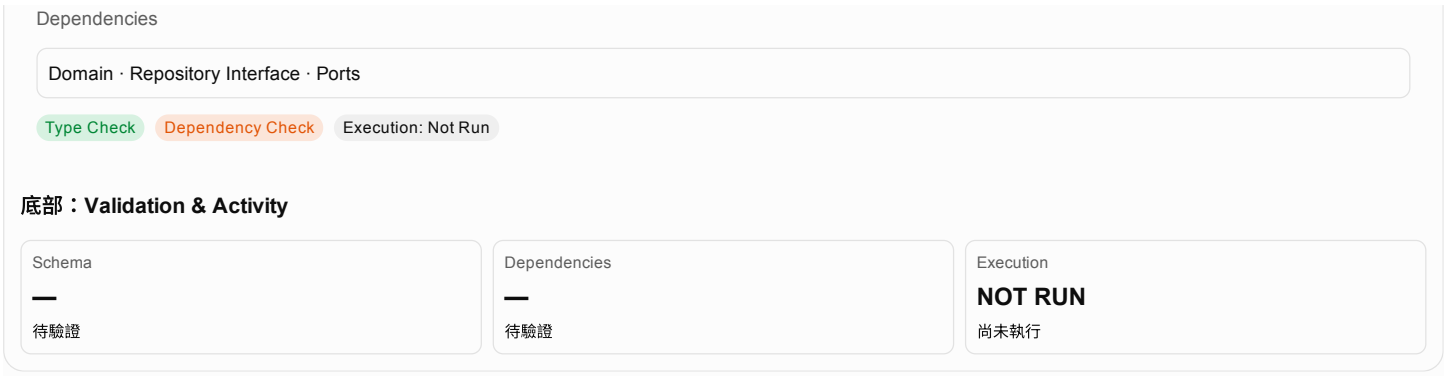
我建議保留你目前的三欄式深色介面，以及 `FLOW-001`、BDD、TDD、Inspector 和底部活動紀錄等既有設計，將 10 大節點的 I/O 規格整合到現有工作流程中，而不是另建一套孤立的管理介面。

核心原則是：

- **規格先於執行**：未通過契約與依賴驗證的節點，不得進入正式執行。
- **型別先於連線**：兩個節點不能只因為都有 Output 和 Input 就直接連接，必須確認型別相容。
- **規格與執行分離**：節點定義、驗證結果、實際執行狀態是三種不同的資料。
- **Agent 受規格約束**：Agent 可以生成或修改實作，但不能自行繞過驗證規則。

一、整合後的 UI 版面





以上是整合後的介面概念。真正實作時，中央工作區應能在「節點卡片、依賴圖、I/O 連線圖」之間切換，右側 Inspector 則始終維持一致的規格編輯與檢查體驗。

二、10 大節點的嚴格 I/O 契約

先統一規格語意：每個節點都必須描述輸入、輸出、依賴、驗證與錯誤；但不是每個節點都必須有執行期 Input/Output。例如 Contract 是型別定義，不是一般的執行節點。

節點	Input 契約	Output 契約	核心驗證規則
`API`	HTTP Request、RPC Message、Webhook Event	HTTP Response、RPC Response、Protocol Error	協議、身分驗證、請求格式、狀態碼
`Contract`	Schema 定義、版本與相容性政策	Type Definition、Schema ID、驗證器	唯一 ID、型別合法、版本相容
`UI`	View Model、User Input、UI Event	User Intent、Command / Query Request	表單驗證、事件格式、權限
`Command`	強型別寫入指令	Result ID、Void、Domain Error 或 Operation Result	前置條件、冪等性、交易與授權
`Query`	Filter、Sort、Pagination、Read Context	Read DTO、Page Result	查詢條件、分頁邊界、讀取權限
`Application`	Use-case Request、執行上下文	Use-case Result、錯誤或事件紀錄	流程完整性、交易邊界、依賴方向
`Domain`	Entity、Value Object、Domain Event	Domain State、Domain Event、Domain Error	業務不變條件、狀態轉移合法性
`Infrastructure`	Repository Port、Persistence Request	還原的 Domain Object、Persistence Result	資料映射、交易、連線與一致性
`ACL`	內部 Port / Contract、外部回應	內部型別結果、標準化錯誤	雙向映射、外部格式隔離、超時
`Custom`	Extension Manifest、Config、Dependency Registry	Plugin Instance、Capability、Lifecycle Result	權限、介面相容性、生命週期與資源隔離

必須修正的四個架構細節

1. API 與 UI 的關係不應被寫死。 UI 可以呼叫 API，也可以直接透過應用程式內部介面呼叫 Application。API 是通訊邊界，不一定是 UI 的唯一入口。

2. Command 與 Application 不一定是兩個執行層。 Command 通常是描述寫入意圖的資料型別；Application Handler 才是協調執行的元件。如果 SGGenAxis 將 `Command` 定義為獨立節點，應明確標記它是 `Message Definition` 還是 `Command Handler`，避免同名異義。

3. Query 可以繞過 Domain，但不是無條件繞過。 查詢可以採用直接投影至 Read DTO 的方式，不必還原完整 Domain Entity；但權限、租戶隔離、資料一致性仍然必須受到控制。

4. Infrastructure 的儲存結果不宜只定義為 Boolean。 `true/false` 無法區分唯一鍵衝突、資料庫斷線、交易失敗等情況。應使用帶有型別的結果，例如 `PersistenceResult<T>`，讓上層能採取正確的處理方式。

三、統一的 Node Schema

SGGenAxis 應為 10 大節點建立共同的 Schema 外殼，再依照節點類別載入專屬規則。

建議每個節點至少具備以下欄位：

欄位	型別	用途
`node_id`	String / UUID	節點唯一識別碼
`node_type`	Enum	10 大節點之一
`schema_version`	SemVer	節點契約版本
`inputs`	Port Definition[]	輸入埠與型別
`outputs`	Port Definition[]	輸出埠與型別
`dependencies`	Dependency[]	節點與模組依賴
`validation_rules`	Rule Reference[]	必須執行的驗證
`execution_policy`	Execution Policy	執行、重試、逾時等政策
`error_contract`	Error Definition[]	可預期的錯誤型別
`metadata`	Object	名稱、描述、標籤等中繼資料

以下是適合 SGGenAxis 的概念性 Schema 範例：

JSON

```
{
  "node_id": "application.change-password",
  "node_type": "Application",
  "schema_version": "1.0.0",
  "inputs": [
    {
      "port_id": "command",
      "direction": "input",
      "data_type": "ChangePasswordCommand",
      "required": true,
      "cardinality": "one"
    }
  ],
  "outputs": [
    {
      "port_id": "result",
      "direction": "output",
      "data_type": "OperationResult",
      "required": true,
      "cardinality": "one"
    }
  ],
  "dependencies": [
    {
      "target": "domain.user",
      "dependency_type": "domain_service"
    },
    {
      "target": "ports.user_repository",
      "dependency_type": "interface"
```

這是供設計討論的示意 Schema，不是已經通過正式 JSON Schema 驗證的成品。實作時還應限制每種 `node_type` 可以使用的欄位，並定義完整的 `OperationResult`、錯誤型別與依賴目標格式。

型別系統不能只比對字串

建議 SGGenAxis 的型別檢查至少區分以下類型：

- **Primitive**：`String`、`Boolean`、`Int32`、`Float64`
- **Structured**：DTO、Entity、Value Object、Schema Object
- **Collection**：`List<T>`、`Map<K,V>`、`Page<T>`
- **Protocol**：HTTP Request、HTTP Response、RPC Message
- **Event**：Domain Event、Integration Event
- **Result**：`Result<T,E>`、`OperationResult<T>`
- **Reference**：Schema Reference、Port Reference、Module Reference

連線時的判斷應包含型別、方向、基數、可空性與版本相容性。即使兩端都叫 `String`，若一端代表未驗證的外部輸入，另一端要求已驗證的密碼憑證，也不能直接視為安全相容。

四、依賴方向與連線驗證規則

SGGenAxis 的依賴圖必須能區分「資料流」與「程式依賴」。這兩種關係不能使用同一套判斷規則。

建議的架構依賴圖



Contract 提供型別契約；Custom 負責擴充元件的組裝與註冊。兩者不應被強迫放在同一條執行管線中。

這張圖代表的是建議的 Clean Architecture / Ports and Adapters 依賴模型，而不是唯一適用的拓樸。實際連線應由節點角色與契約決定。

SGGenAxis 必須執行的規則

規則 ID	驗證項目	失敗條件	嚴重度
`IO-001`	Port 方向	Input 接 Input，或 Output 接 Output	Error
`IO-002`	型別相容	型別不相容且沒有合法轉換器	Error
`IO-003`	必填輸入	必要 Input 沒有來源或合法預設值	Error
`IO-004`	基數限制	單一輸入接收到不允許的多個來源	Error
`IO-005`	版本相容	Schema 版本不符合相容性政策	Error
`ARCH-001`	依賴方向	Domain 直接依賴 Infrastructure 實作	Error
`ARCH-002`	循環依賴	禁止循環的模組依賴圖出現環路	Error
`ARCH-003`	層級邊界	Domain 直接引用 UI DTO 或 ORM Model	Error
`ARCH-004`	契約完整性	Port 引用不存在或未註冊的 Schema	Error
`SEC-001`	權限與資料界線	缺少必要授權或租戶隔離	Error
`RUN-001`	執行前檢查	未通過必要驗證便要求正式執行	Blocker

注意：循環檢查應作用於需要維持無環的依賴圖；某些事件驅動系統允許執行期回饋或訊息循環，不能一律禁止所有圖形循環。

五、Inspector 的完整設計

點選任一節點，右側 Inspector 應顯示六個分頁。這樣規格、驗證、實作與執行狀態才不會混在一起。



`command` · `ChangePasswordCommand`

Required · Cardinality: One · Schema Ref: 必須存在

Outputs 1 Port

`result` · `OperationResult`

Result Schema · Success / Typed Failure

以上是 Application 範例。切換節點時，應讀取該節點實際的 Schema，而不是所有類別都套用相同欄位。

六、錯誤處理與狀態機

SGGenAxis 應分別管理「規格狀態」、「驗證狀態」及「執行狀態」，而不是將三者濃縮成一個綠燈。

維度	建議狀態	語意
規格	`DRAFT` 、 `REVIEW` 、 `APPROVED` 、 `DEPRECATED`	規格的生命週期
驗證	`NOT_RUN` 、 `RUNNING` 、 `PASSED` 、 `FAILED` 、 `STALE`	最近一次驗證結果
執行	`NOT_RUN` 、 `QUEUED` 、 `RUNNING` 、 `SUCCEEDED` 、 `FAILED` 、 `CANCELLED`	真正的工作執行結果

建議執行前條件為：

$$\text{CanExecute} = \text{Approved} \wedge \text{ValidationPassed} \wedge \text{ValidationFresh} \wedge \text{DependenciesResolved}$$

另外，還要通過執行環境、權限與資源檢查。上式是 SGGenAxis 的建議政策，不代表所有節點都需要執行；純 Contract 定義通常只需要驗證與發布。

錯誤物件格式

所有驗證器與執行器應輸出統一錯誤結構，例如：

JSON

```
{
  "error_id": "IO-002",
  "severity": "ERROR",
  "category": "TYPE_MISMATCH",
  "node_id": "application.change-password",
  "port_id": "command",
  "message": "Input type is incompatible with the connected output.",
  "expected_type": "ChangePasswordCommand",
  "actual_type": "ChangePasswordRequest",
  "source_ref": "api.change-password",
  "suggested_action": "Add an explicit mapping adapter.",
  "retryable": false
}
```

這樣 Inspector 可以直接定位問題節點與 Port，依賴圖可以將連線標紅，底部活動紀錄則能顯示時間、規則 ID、錯誤內容及修正紀錄。

對 Agent 而言，這個結構也很重要：Agent 收到的不是模糊的「執行失敗」，而是可以定位、修正、重新驗證的具體規格錯誤。

七、Agent 的執行流程

01 讀取規格
載入 Node Schema、版本與依賴

↓

02 靜態驗證
檢查型別、Port、引用與依賴方向

↓

03 產生變更計畫
列出預計修改檔案、風險與測試

04 **授權與執行**
依政策執行，限制工具、權限與範圍

05 **測試與回報**
執行測試、收集結果與錯誤

06 **產生驗收證據**
保存版本、日誌、測試結果與差異

Agent 不能自行修改已核准的 Schema 後直接執行。若它需要更動契約，應建立新的變更版本、重新驗證，並依照審核政策取得授權。

八、建議的開發優先順序

不要一開始就開發完整的可視化節點編輯器。先讓底層規格與驗證結果可靠，再將它們呈現到 UI。

1. **第一階段：Schema Registry。** 定義 10 大節點、Port 型別、版本與錯誤模型。
2. **第二階段：Validation Engine。** 實作型別相容、必填輸入、依賴方向、引用完整性與無環依賴檢查。
3. **第三階段：Inspector。** 將 I/O、依賴、驗證、實作綁定與執行狀態整合到既有 UI。
4. **第四階段：Dependency Graph。** 支援節點連線、錯誤定位、影響範圍分析與規格差異比較。
5. **第五階段：Agent Execution。** 將驗證結果、變更計畫、測試證據與審核流程串接起來。

第一階段的驗收標準

- 10 大節點均有明確且版本化的 Schema。
- 不合法的型別連線能被拒絕，並指出實際的 Input、Output 與錯誤原因。
- 不存在的 Schema、模組或 Port 不能被當成有效引用。
- 規格變更會使受影響的驗證結果失效。
- Domain 不會因為連線或自動生成而直接依賴 Infrastructure 實作。
- Agent 在驗證未通過或授權不足時不能啟動受保護的正式執行。
- UI 能區分「未驗證」、「驗證失敗」、「驗證通過」與「程式執行成功」。

最終建議： SGGenAxis 應以 `Node Schema → Validation Engine → Inspector → Dependency Graph → Agent Execution` 為核心架構。這樣 UI 不只是把節點畫出來，而是能根據正式的型別契約與驗證結果，真正管理整套系統的工程規格與執行流程。